

OPISKELIJAN OHJATTU TYÖPAKETTI

Koulun kahvilapeli

Näyttöprojekti: Ohjelmointi, 45 osp + Ohjelmistokehittäjänä toimiminen, 45 osp

ALOITUS vko 34 17.8.2026	SYYSLOMA vko 42 ei projektityötä	JULKAISU vko 48 Unity WebGL / GitHub Pages	VALMIS vko 49 pe 4.12.2026
---------------------------------------	---	--	---

Opiskelija: _____

Ryhmä: _____ Ohjaaja / arvioijat: _____

Asiakas / toimeksiantaja: _____

Projektin tavoite: tee pieni, valmis, testattu ja julkaistu peli. Näytössä tärkeintä ei ole pelin koko vaan se, että työskentely, ohjelmointiratkaisut, palaute, testaus ja julkaisu ovat näkyvissä ja perusteltavissa.

Näin käytät tätä työpakettia

1. Lue ensin toimeksianto ja kirjoita asiakkaalle kysymykset. Älä päättää avoimia vaatimuksia yksin.
2. Merkitse viikkoaikatauluun valmistuneet asiat ja näytön todisteet joka perjantai.
3. Täytä erillistä Näytön dokumentointipohjat -työpakettia työn aikana, ei vasta lopussa.
4. Kokoa lopuksi näyttöaineisto: julkaistu peli, repository, katselmoinnit, testit, dokumentit ja itsearviointi.

Valittu peli-idea

Koulun kahvilapeli. Pelaaja ottaa vastaan 1-3 tuotteen tilauksia ja toimittaa ne oikein ennen ajan loppumista. Pieni ydinsilmukka mahdollistaa kaikki näytössä tarvittavat osa-alueet: käyttöliittymän, toimintalogiikan, JSON-datan, tallennuksen, Gitin, testauksen, asiakaspalautteen ja julkaisun.

1. Asiakkaan toimeksianto ja projektin raja

TOIMEKSIANTO OPISKELIJALLE

Toteuta Unity 2D + C# -projektina selaimessa toimiva koulun kahvilapeli. Asiakkaita saapuu tiskille ja he tilaavat 1-3 tuotetta. Pelaajan tehtävä on toimittaa oikea tilaus mahdollisimman nopeasti.

Sovittu toteutustapa: Unity 2D + C#, yksi CafeGame-scene, Canvas-paneelit, products.json TextAssetina, top 5 PlayerPrefsissa ja WebGL-julkaisu GitHub Pagesiin. Käytä samaa oppilaitoksen määrittämää Unity-versiota koko projektin ajan.

Pelissä pitää olla aloitusvalikko, pelinäköymä, pistelasku ja pelin päättymisnäköymä. Vaikeustason pitää kasvaa pelin edetessä. Pelaajan parhaat tulokset tallennetaan. Tuotetietoja ei saa kovakoodata pelilogiikkaan, vaan ne luetaan erillisestä tietolähteestä, esimerkiksi JSON-tiedostosta. Asiakkaalle esitellään toimiva versio ennen lopullista julkaisua, ja palautteen perusteella tehdään perusteltu muutos. Lopullinen peli julkaistaan niin, että asiakas voi kokeilla sitä.

Tärkeä raja: älä lisää ominaisuutta vain siksi, että se kuulostaa hienolta. Pakollinen, toimiva kokonaisuus tehdään ensin. Vasta sen jälkeen voidaan lisätä esimerkiksi pause, vaikeustasot tai ääniä. Verkkomonipeli, käyttäjätilit, oikeat maksut ja laaja 3D-maailma eivät kuulu projektiin.

Kysy asiakkaalta ennen toteutusta

- Millä selaimilla ja laitteilla Unity WebGL -versio testataan, ja millä tavalla peliä ohjataan?
- Mitkä tuotteet kahvilassa ovat ja kuinka monta tuotetta käytetään ensimmäisessä versiossa?
- Kuinka monta asiakasta voi olla näkyvissä ja miten uusi asiakas saapuu?
- Miten vaikeustaso kasvaa: nopeampi aika, useampi tuote, tiheimmät asiakkaat vai jokin yhdistelmä?
- Miten pisteet lasketaan ja mitä tapahtuu väärästä toimituksesta?
- Millä nimimerkkisäännöillä ja tasatilanteen järjestyksellä viiden parhaan tuloksen lista toteutetaan?
- Millainen käyttöliittymä sopii kohderyhmälle ja mitä tietoa pelin aikana pitää aina nähdä?
- Miten ja milloin asiakas haluaa kokeilla ensimmäistä pelattavaa versiota?

Valmis kun - pakollinen perusversio

- Aloitusvalikko, pelinäköymä ja lopetus-/tulostäköymä toimivat.
- Asiakas tilaa 1-3 tuotetta, ja peli tunnistaa oikean sekä väärän toimituksen.
- Ajastin, pisteet ja pelin päättymisen toimivat sovittujen sääntöjen mukaan.
- Vaikeus kasvaa pelin edetessä asiakkaan kanssa sovitulla tavalla.
- Tuotteiden nimet, pistearvot tai muut sovitut tiedot luetaan JSON-tiedostosta.
- Viiden parhaan tuloksen lista tallentuu ja latautuu uudelleen pelin käynnistyessä.
- Käyttöliittymä vastaa mockupia tai poikkeamat on perusteltu.
- Projektissa on järkevä koodirakenne, jatkuva Git-historia ja vähintään yksi yhdistetty feature-branch.
- Asiakas on nähnyt kaksi toimivaa versiota, ja palaute sekä päätetyt muutokset on kirjattu.
- Peli on testattu, tunnetut puutteet on kirjattu ja versio 1.0 on julkaistu.

2. Projekti käyntiin - ensimmäiset viisi työpäivää

1. **Lue toimeksianto ja merkitse avoimet asiat.** Kirjoita vähintään kahdeksan kysymystä asiakkaalle. Merkitse oletukset erikseen.
Näytön todiste: Kysymyslista ja päivätty muistiinpano.
2. **Pidä aloituskeskustelu.** Kysy kohderyhmä, alusta, pelisäännöt, tallennettava tieto ja ensimmäisen katselmoinnin ajankohta. Selitä asiakkaalle vaihtoehdot tavallisella kielellä.
Näytön todiste: Asiakastarve ja päätökset dokumentointipohjan osassa 2.
3. Käytä oppilaitoksen määrittämää Unity-versiota Unity Hubissa. Luo 2D-projekti ja CafeGame-scene. Varmista heti, että tyhjästä projektista voidaan tehdä toimiva WebGL-testibuild.
Näytön todiste: Kuvakaappaus editorista ja toimivasta testiversiosta.
4. **Perusta Git-repository ennen varsinaista koodausta.** Lisää oikea .gitignore, README ja docs-kansio. Tee ensimmäinen commit ja varmista, että repository näkyy etäpalvelussa.
Näytön todiste: Repository-linkki ja ensimmäinen commit.
5. **Täytä näyttöprojektin suunnitelman ensimmäinen versio.** Kirjaa tavoite, käyttäjäryhmä, teknologiat, oma rooli, tärkeimmät ominaisuudet, prioriteetit ja valmis kun -ehdot.
Näytön todiste: Dokumentointipohjan osa 1 valmis asiakkaan tarkistettavaksi.
6. **Tee backlog.** Pilkkoo työ 0,5-1 päivän tehtäviksi. Merkitse P0 = pakollinen, P1 = tärkeä ja P2 = lisäominaisuus. Arvioi työmäärä ja nimeä tekijä.
Näytön todiste: GitHub Issues-, Trello- tai muu näkymä sekä linkki osassa 3.
7. **Piirrä käyttöliittymän mockup ennen toteutusta.** Näytä aloitusvalikko, pelinäkymä, tilaus, aika, pisteet ja lopetusnäkymä. Pyydä asiakkaalta hyväksyntä.
Näytön todiste: Vähintään yksi päivätty mockup-kuva osassa 4.
8. **Suunnittele koodin vastuut.** Eroti vähintään pelitila, tilausten muodostus, tuotetiedon lataus, pisteet/ajastin, tallennus ja käyttöliittymä omiksi vastuiksi.
Näytön todiste: Lyhyt komponenttilista tai kaavio osassa 4.
9. **Tee ensimmäinen pystyleikkaus.** Rakenna polku aloituksesta yhteen tilaukseen, oikeaan toimitukseen, pisteeseen ja pelin päättymiseen. Grafiikka saa olla väliaikaista.
Näytön todiste: Toimiva build, video tai kuvakaappaus.
10. **Sovi viikkorytmi.** Varaa viikoittainen ohjaus, asiakkaan katselmointi viikolle 41 ja julkaisuehdokkaan katselmointi viikolle 47.
Näytön todiste: Kalenterimerkinnät ja nimetyt osallistujat.

3. Työrutiini, Git ja yhteistyö

Yksi työjakso: valitse pieni tehtävä, suunnittele, toteuta, testaa, tallenna Gitillä ja kirjaa mitä opit. Jos et pysty selittämään muutosta, työ ei ole vielä valmis.

Jokaisen työskentelykerran kuusi vaihetta

1. Valitse backlogista yksi tehtävä ja lue sen valmis kun -ehdot.
2. Hae uusin toimiva versio ja tee tarvittaessa oma feature-branch, esimerkiksi feature/order-system.
3. Kirjoita lyhyt suunnitelma: mitä tiedostoja tai osia muutan ja miten testaan muutoksen?
4. Toteuta pieni osa kerrallaan. Käynnistä peli usein ja seuraa virheilmoituksia sekä lokia.
5. Testaa sekä tavallinen käyttö että vähintään yksi poikkeustilanne. Kirjaa bugit testauslokiin.
6. Tee selkeä commit ja push. Päivitä tehtävän tila, dokumentointi ja mahdollinen tekoälyn käyttöloki.

Git-käytännöt

- ☐ main sisältää vain toimivan version; keskeneräinen ominaisuus tehdään branchissa.
- ☐ Yksi commit kertoo yhden ymmärrettävän muutoksen. Esimerkki: Add JSON product loader and validation.
- ☐ Commitoi vähintään jokaisena aktiivisena työpäivänä, kun on syntynyt ehjä muutos.
- ☐ Yhdistä vähintään yksi feature-branch mainiin katselmoinnin tai pull requestin kautta.
- ☐ Älä lisää repositoryyn salasanoja, API-avaimia, välimuistia tai turhia build-tiedostoja.

Yhteistyö myös yksilöprojektissa

Jos teet pelin yksin, opettaja, asiakas ja nimetty vertaisopiskelija muodostavat kehitystiimin ympärillesi. Sovi tehtävistä ohjaajan kanssa, vertaile vähintään kerran kahta ratkaisuvaihtoehtoa toisen henkilön kanssa ja pyydä koodin tai toimivan version katselmointi. Kirjaa osallistujat, päätökset ja oma roolisi.

Viikon lopun viisi pakollista merkintää

- ☐ toimiva build tai selkeä tieto siitä, mikä estää buildin
- ☐ päivitetty backlog ja toteutuneen työmäärän huomio
- ☐ Git-commitit ja tarvittavat branch-/merge-tiedot
- ☐ vähintään yksi testaus- tai virheenkorjausmerkintä
- ☐ näytön todiste: linkki, kuvakaappaus, testitulos, päätösmuistio tai demo

4. Tekoälyn sallittu ja vastuullinen käyttö

Tekoäly on sallittu apuväline. Saat käyttää sitä ideointiin, suunnitteluun, koodin selittämiseen, virheiden tutkimiseen, testitapausten ehdottamiseen ja dokumentoinnin kielen parantamiseen. Sinä olet silti jokaisen projektiin jäävän ratkaisun tekijä ja vastuuhenkilö.

Neljä sääntöä

1. **Ymmärrä:** pysty selittämään omin sanoin, mitä ehdotettu koodi tekee ja miksi se sopii projektiin.
2. **Tarkista:** vertaa tarvittaessa moottorin tai kirjaston dokumentaatioon. Tekoäly voi keksiä olemattomia metodeja.
3. **Testaa:** älä merkitse ominaisuutta valmiiksi ennen kuin olet itse ajanut testit ja tarkistanut tulokset.
4. **Kirjaa:** merkitse merkittävä tekoälyn käyttö lokiin sekä kerro, mitä muutit, hylkäsit tai opit.

Hyviä käyttötapoja

- Pyydä ensin selitys tai ratkaisuvaihtoehdot, ei kokonaista valmista ominaisuutta.
- Anna pieni, rajattu koodikatkkelma ja tarkka virheilmoitus. Poista henkilötiedot ja salaisuudet.
- Pyydä tekoälyä laatimaan testitapauksia, reunatapauksia tai kysymyksiä, joiden avulla itse ratkaiset ongelman.
- Pyydä koodikatselmointi: nimeä toisto, epäselvät vastuut ja mahdolliset virhetilanteet.
- Käytä tekoälyä tekstin selkeyttämiseen vasta sen jälkeen, kun faktat, päätökset ja oppiminen ovat omiasi.

Esimerkkikehotteita

- Selitä tämä virheilmoitus aloittelijalle. Anna kolme todennäköistä syytä ja tarkistukset, mutta älä kirjoita valmista ratkaisua.
- Ehdota 8 testitapausta tilauslogiikalle. Lisää odotettu tulos ja vähintään kolme reunatapauksia.
- Vertaa kahta tapaa erottaa tilausten muodostus käyttöliittymästä. Kerro hyödyt, riskit ja mitä minun pitäisi itse päättää.
- Katselmoi tämä metodi ylläpidettävyyden kannalta. Älä muuta toimintaa. Perustele jokainen ehdotus.

EI HYVÄKSYTTÄVÄÄ

Älä palauta koodia, jota et ymmärrä; älä keksi testituloksia, palautetta tai tapaamisia; älä syötä tekoälylle salasanoja, API-avaimia tai tarpeettomia henkilötietoja; älä anna tekoälyn kirjoittaa itsearviointia puolestasi.

Tekoälyn käyttöloki

Täytä yksi rivi aina, kun tekoäly vaikutti projektiin jäävään työhön. Lisää todisteviite ja varmista, ettet syöttänyt henkilötietoja, salaisuuksia tai luottamuksellista aineistoa.

[illegible]

JAKSON LOPUSSA

Hyödyllisin tekoälyltä saamani apu: _____

Ehdotus, jonka hylkäsin tai muutin: _____

Ratkaisu, jonka osaan selittää näytössä: _____

5. Paperinen viikkoaikataulu

Merkitse jokaisen viikon lopussa tila ja kirjoita ohjaajan huomio. Viikko 42 on syysloma, eikä sille ole suunniteltu projektityötä. Jos viikko jää kesken, siirrä vain tärkein keskeneräinen tehtävä seuraavalle viikolle ja pidä perusversion rajausta ennallaan.

Vaihe A - tarve, suunnitelma ja ensimmäinen pelattava polku

VKO 34 | 17.-21.8. Käynnistys ja asiakastarve

TEHTÄVÄT	☐ Lue toimeksianto, laadi vähintään 8 kysymystä ja pidä aloituskeskustelu. ☐ Asenna sovittu Unity-versio Unity Hubissa, luo 2D-projekti ja varmista WebGL-testibuild. ☐ Perusta repository, README, .gitignore ja docs-kansio.
VIIKON NÄYTTÖ	Asiakastarpeen muistutinpano, repository-linkki, ensimmäinen commit ja toimiva testibuild.

Opiskelija: ☐ valmis ☐ kesken Ohjaajan kuittaus / huomio: _____

VKO 35 | 24.-28.8. Rajausta ja suunnitelmat

TEHTÄVÄT	☐ Täytä dokumentointipohjan osat 1-4 ensimmäiseen versioon. ☐ Tee P0/P1/P2-backlog, arviot ja valmis kun -ehdot. ☐ Piirrä UI-mockup ja suunnittele Unityn CafeGame-scene, Canvas-paneelit, C#-vastuut, JSON sekä tallennus.
VIIKON NÄYTTÖ	Hyväksytty rajausta, päivätty mockup, tekninen suunnitelma ja backlog-linkki.

Opiskelija: ☐ valmis ☐ kesken Ohjaajan kuittaus / huomio: _____

VKO 36 | 31.8.-4.9. Ensimmäinen pystyleikkaus

TEHTÄVÄT	☐ Tee CafeGame-scenen Canvasille MenuPanel, GamePanel ja ResultPanel väliaikaisella grafiikalla. ☐ Toteuta yksi asiakas, yksi tilaus, oikea toimitus ja piste. ☐ Kirjaa ensimmäiset testit ja bugit.
VIIKON NÄYTTÖ	Pelattava polku alusta loppuun, video/kuva ja vähintään 2 testimerkintää.

Opiskelija: ☐ valmis ☐ kesken Ohjaajan kuittaus / huomio: _____

VKO 37 | 7.-11.9. Tuotetiedot ja tilauslogiikka

TEHTÄVÄT	☐ Luo products.json ja lataa se Unityn TextAsset-viitteenä C#-olioiksi JsonUtilitylla; käsittele puuttuva ja rikkinäinen data. ☐ Toteuta 1-3 tuotteen tilaukset ja oikean/väärän toimituksen tarkistus. ☐ Pidä OrderManagerin C#-logiikka erillään UIControllerista ja dokumentoi vastuut.
VIIKON NÄYTTÖ	JSON-esimerkki, latauslogiikan commit, testit tyhjälle/virheelliselle datalle.

Opiskelija: ☐ valmis ☐ kesken Ohjaajan kuittaus / huomio: _____

Vaihe B - ydintoiminnot, tallennus ja ensimmäinen katselmointi

VKO 38 | 14.-18.9. Pisteet, aika ja palaute

TEHTÄVÄT	☐ Toteuta C#:lla pisteytys, Time.deltaTime-ajastin ja pelin päättymisehdot. ☐ Näytä Unityn TextMeshPro-kentissä tilaus, aika, pisteet sekä onnistumisen tai virheen palaute. ☐ Testaa rajatapaukset: aika 0, nopea kaksoisklikkaus ja väärä tuote.
VIIKON NÄYTTÖ	Toimiva pelisilmukka, 3 testitapausta ja yksi dokumentoitu bugikorjaus.

Opiskelija: ☐ valmis ☐ kesken Ohjaajan kuittaus / huomio: _____

VKO 39 | 21.-25.9. Vaikeuden kasvu ja pelitilat

TEHTÄVÄT	☐ Toteuta sovittu vaikeuden kasvu ja pidä arvot helposti säädettävänä. ☐ Tee selkeät tilat: valikko, peli, tauko tarvittaessa ja tulos. ☐ Vertaile ohjaajan/vertaisen kanssa kahta toteutusvaihtoehtoa ja kirjaa päätös.
VIIKON NÄYTTÖ	Ratkaisuvaihtoehtojen muistiinpano, vaikeuslogiikan commit ja hyväksytyt testit.

Opiskelija: ☐ valmis ☐ kesken Ohjaajan kuittaus / huomio: _____

VKO 40 | 28.9.-2.10. Tallennus, tietovarasto ja tietoturva

TEHTÄVÄT	☐ Tallenna ja lataa Unityn PlayerPrefsilla viiden parhaan tuloksen ScoreList JSON-merkkijonona. ☐ Perustele dokumentointipohjan osassa 4, miksi top 5 tallennetaan JSON-merkkijonona PlayerPrefsiin ja mitkä ovat ratkaisun rajat. ☐ Validoi luettu data ja nimimerkki; älä tallenna salasanoja tai API-avaimia.
VIIKON NÄYTTÖ	Tallennus toimii uudelleenkäynnistyksen jälkeen, riskit ja ratkaisut on kirjattu.

Opiskelija: ☐ valmis ☐ kesken Ohjaajan kuittaus / huomio: _____

VKO 41 | 5.-9.10. Ensimmäinen asiakaskatselmointi

TEHTÄVÄT	☐ Tee asiakkaalle kokeiltava build ja valmistelee 5-10 minuutin demo. ☐ Kerää palaute, selitä tekniset ratkaisut ymmärrettävästi ja sovi yksi rajattu muutos. ☐ Päivitä backlog, prioriteetit, arviot ja katselmointiloki.
VIIKON NÄYTTÖ	Katselmointiloki, palaute, päätetty muutos, päivitetty backlog ja toimiva build.

Opiskelija: ☐ valmis ☐ kesken Ohjaajan kuittaus / huomio: _____

VKO 42 | 12.-16.10. - SYSSLOMA

Ei projektityötä. Aikataulu ei edellytä koodausta, dokumentointia tai korvaavaa työskentelyä syyslomalla. Jatka viikolla 43 viimeisimmästä toimivasta main-versiosta.

Vaihe C - palaute, viimeistely ja laadunvarmistus

VKO 43 | 19.-23.10. Palautemuutos branchissa

TEHTÄVÄT	☐ Muuta asiakkaan palaute pieniksi tehtäviksi ja arvioi työmäärä. ☐ Toteuta sovittu muutos omassa feature-branchissa. ☐ Katselmoi, testaa ja yhdistä muutos toimivaan main-versioon.
VIIKON NÄYTTÖ	Issue-linkki, branch/merge tai pull request, testit ja päivitetty katselmointiloki.

Opiskelija: ☐ valmis ☐ kesken Ohjaajan kuittaus / huomio: _____

VKO 44 | 26.-30.10. Käyttöliittymä ja käytettävyys

TEHTÄVÄT	☐ Vertaa toteutusta mockupiin ja kirjaa perustellut poikkeamat. ☐ Paranna luettavuutta, palautetta, näppäimistö-/hiiriohjausta ja virhetilanteita. ☐ Pyydä vertaiselta lyhyt käytettävyystesti.
VIIKON NÄYTTÖ	Ennen/jälkeen-kuvat, vertaispalaute, tehdyt päätökset ja korjaukset.

Opiskelija: ☐ valmis ☐ kesken Ohjaajan kuittaus / huomio: _____

VKO 45 | 2.-6.11. Järjestelmällinen testaus

TEHTÄVÄT	☐ Laadi vähintään 12 testitapausta: normaali käyttö, rajatapaukset ja tallennus. ☐ Kirjaa kaikki aidot havainnot. Osoita 3 täydellistä virheenkorjausketjua aidoista havainnoista tai opettajan merkitsemistä vikatehtävistä. ☐ Testaa puhdas käynnistys sekä puuttuva/virheellinen JSON- tai tallennustiedosto.
VIIKON NÄYTTÖ	Testausloki, testitulokset, bugikorjaukset ja uusintatestit.

Opiskelija: ☐ valmis ☐ kesken Ohjaajan kuittaus / huomio: _____

VKO 46 | 9.-13.11. Koodin laatu ja tekninen katselmointi

TEHTÄVÄT	☐ Poista turha toisto, nimeä muuttujat/metodit selkeästi ja rajaa vastuut. ☐ Pyydä koodikatselmointi ohjaajalta/vertaiselta ja kirjaa palaute. ☐ Varmista, että osaat selittää myös tekoälyn avulla syntyneet ratkaisut.
VIIKON NÄYTTÖ	Katselmointimuistio, refaktorointicommit ja perustelu rakenteelle.

Opiskelija: ☐ valmis ☐ kesken Ohjaajan kuittaus / huomio: _____

Vaihe D - julkaisuehdokas, julkaisu ja näyttö

VKO 47 | 16.-20.11. Julkaisuehdokas ja toinen katselmointi

TEHTÄVÄT	☐ Tee release candidate ja jäädytä uusien ominaisuuksien lisääminen. ☐ Anna asiakkaan ja vähintään yhden muun käyttäjän testata peli. ☐ Luokittele palaute: korjataan ennen julkaisua / tunnettu puute / ei tähän versioon.
VIIKON NÄYTTÖ	RC-build, testipalautteet, päätökset ja viimeinen korjauslista.

Opiskelija: ☐ valmis ☐ kesken Ohjaajan kuittaus / huomio: _____

VKO 48 | 23.-27.11. Versio 1.0 ja julkaisu

TEHTÄVÄT	☐ Korjaa vain julkaisemisen estävät ja selvästi tärkeät virheet. ☐ Tee puhdas Unity WebGL -build docs-kansioon, lisää v1.0-tagia ja julkaise GitHub Pagesiin. ☐ Testaa julkaistu linkki toisella laitteella tai selaimella ja kirjoita käyttöohje.
VIIKON NÄYTTÖ	Toimiva julkaisulinkki, v1.0/tagia, käyttöohje ja tunnetut puutteet.

Opiskelija: ☐ valmis ☐ kesken Ohjaajan kuittaus / huomio: _____

VKO 49 | 30.11.-4.12. Dokumentointi, itsearviointi ja luovutus

TEHTÄVÄT	☐ Täytä dokumentointipohjan osat 7-8 ja tarkista kaikki aiemmat osat. ☐ Täytä osaamisvaatimusten näyttömatrissiin linkit tai todisteet. ☐ Harjoittele demo ja varaudu selittämään koodi, bugi, tallennus, Git ja tekoälyn käyttö. ☐ Luovuta viimeistään perjantaina 4.12.2026.
VIIKON NÄYTTÖ	Peli, repository, valmis työpaketti, näyttömatrissi, AI-loki ja esitysdemo.

Opiskelija: ☐ valmis ☐ kesken Ohjaajan kuittaus / huomio: _____

6. Katselmoinnit, testaus ja julkaisu

Neljä ohjauspistettä

Aika	Katselmointi	Näytä	Kirjaa päätös
Vko 35	Suunnitelmakatselmoi nti	Tarve, rajaus, mockup, backlog, tekninen suunnitelma	Asiakas hyväksyy perusversion tavoitteen ja prioriteetit.
Vko 41	Ensimmäinen pelattava versio	Pelattava ydinsilmukka, JSON, pisteet/aika, tallennuksen tilanne	Palaute ja yksi rajattu muutos kirjataan sekä priorisoidaan.
Vko 47	Julkaisuehdokas	RC-build, testit, tunnetut puutteet, julkaisutapa	Päätös: korjataan / hyväksytään tunnetuksi puutteeksi / siirretään.
Vko 49	Lopullinen näyttö	Julkaistu peli, repository, dokumentit, todisteet ja itsearviointi	Opiskelija osoittaa ymmärtävänsä ratkaisut ja työprosessin.

Testauksen vähimmäistavoite

- ☐ Vähintään 12 suunniteltua testitapausta, joissa näkyvät odotettu tulos ja toteutunut tulos.
- ☐ 3 täydellistä virheenkorjausketjua aidosta havainnoista tai opettajan merkitsemistä vikatehtävistä.
- ☐ Testit kattavat valikot, tilauksen, pisteet, ajastimen, vaikeuden, tallennuksen ja pelin päättymisen.
- ☐ Reunatapauksissa tarkistetaan ainakin puuttuva/virheellinen JSON, tyhjä tallennus, ajan loppuminen ja väärä toimitus.
- ☐ Julkaistu Unity WebGL -versio testataan GitHub Pages -linkistä toisella selaimella tai laitteella, ei vain Unity Editorin Play Modessa.

Julkaisun portti viikolla 48

- ☐ main-branch rakentuu ilman projektin estävää virhettä
- ☐ version numero on 1.0 ja repositoryssa on julkaisutagi tai release
- ☐ uusi käyttäjä ymmärtää käynnistyksen ja ohjauksen käyttöohjeesta
- ☐ tulosten tallennus toimii julkaisupaketissa
- ☐ julkaisulinkki toimii toisella laitteella tai selaimella
- ☐ tunnetut puutteet ja rajaukset on kirjattu rehellisesti

7. Ohjelmointi - näytön todisteet

Täytä viimeinen sarake linkillä, tiedostonimellä, commitilla, kuvalla tai dokumentointipohjan kohdan numerolla. Pelkkä valmis peli ei todista työprosessia.

Ammattitaitovaatimus	Mitä teet tässä projektissa	Todiste / oma viite	OK
Käyttää ohjelmointieditoria tai kehitysympäristöä	Työskentele sovitussa Unity-versiossa ja tee WebGL-build itse.	Editorikuva, build, repository	☐
Etsii ja korjaa virheitä ohjelmakoodista	Käytä Unity Consolea tai debuggeria ja selitä 3 täydellistä korjausketjua aidoista havainnoista tai merkityistä vikatehtävistä.	Testaus- ja bugiloki	☐
Testaa ohjelman toimintoja	Suunnittele ja suorita vähintään 12 testiä sekä uusintatetit.	Testiloki ja tulokset	☐
Käyttää rakenteista ohjelmointia toteutuksissa	Jaa tilaukset, pelitila, UI, data ja tallennus järkeviin kokonaisuuksiin.	Koodirakenne ja katselmointi	☐
Kirjoittaa ylläpidettävää ohjelmakoodia	Käytä selkeitä nimiä, pieniä metodeja ja vältä toistoa.	Refaktorointicommit	☐
Tulkitsee suunnitelmia ja toteuttaa käyttöliittymän tai sen osia	Tee mockup ennen UI:ta ja vertaa toteutusta suunnitelmaan.	Mockup + ennen/jälkeen-kuvat	☐
Tulkitsee suunnitelmia ja toteuttaa ohjelmiston toimintoja	Kirjaa user storyt/valmis kun -ehdot ja toteuta niiden mukaan.	Issue-linkit ja commitit	☐
Sopii tehtävistä tiimin muiden jäsenten kanssa	Sovi omat tehtävät ja vastuut ohjaajan/tiimin kanssa.	Tehtävälista ja kokousmuistio	☐
Etsii ratkaisuvaihtoehtoja ja ratkoo ongelmia yhdessä tiimin kanssa	Vertaa kahta teknistä tapaa ja tee yhteinen päätös.	Vaihtoehtomuistio	☐
Arvioi ratkaisujen toimivuuden yhdessä tiimin kanssa	Katselmoi peliä tai koodia toisen kanssa ja reagoi palautteeseen.	Katselmointiloki	☐
Arvioi omaa toimintaa tiimin jäsenenä	Arvioi yhteistyö, avun tarve, onnistumiset ja seuraava kehitysaskel.	Itsearviointi, osa 8	☐

8. Ohjelmistokehittäjänä toimiminen

Ammattitaitovaatimus	Mitä teet tässä projektissa	Todiste / oma viite	OK
Selvittää kehitystiimin kanssa asiakkaan tarpeet	Kysy avoimet vaatimukset ja varmista kohderyhmä, WebGL-testiselaimet sekä tavoite.	Kysymykset + muistio	☐
Viestii tekniset asiat asiakaslähtöisesti	Selitä vaihtoehdot, työmäärä ja rajat ilman tarpeetonta jargonia.	Asiakkaan yhteenveto	☐
Osallistuu version katselmointiin	Esittele toimiva versio viikoilla 41 ja 47 ja käsittele palaute.	Katselmointiloki	☐
Asettaa kehitystiimin kanssa toteutettavat toiminnot tärkeysjärjestykseen	Käytä P0/P1/P2-prioriteetteja ja suojaa perusversion rajaus.	Backlog ennen/jälkeen	☐
Jakaa kehitystiimin kanssa toteutettavat toiminnot tehtäviksi	Pilko ominaisuuksia 0,5-1 päivän issueiksi ja nimeä tekijä.	Issue-lista	☐
Suunnittelee ja arvioi kehitystiimin kanssa tehtävien toteuttamista	Arvioi työmäärä, seuraa toteumaa ja muuta suunnitelmaa perustellusti.	Arvio vs. toteuma	☐
Kehittää ohjelmiston toimintalogiikkaa	Toteuta tilaus, toimitus, pisteet, aika, vaikeus ja pelitilat.	Koodi, demo ja testit	☐
Valitsee ohjelmistoon sopivan tietovaraston	Käytä tuotteisiin TextAsset + JSON -ratkaisua ja top 5 -listaan PlayerPrefsiin tallennettua JSON-merkkijonoa; perustele sopivuus.	Tekninen suunnitelma	☐
Toteuttaa yhteyden tietovarastoon	Lue products.json JsonUtilitylla ja tallenna/lataa top 5 PlayerPrefsista.	Toimiva demo + testit	☐
Hyödyntää rajapintoja ja käsittelee tietoa	Käytä Unityn TextAsset-, JsonUtility- ja PlayerPrefs-rajapintoja; muuta data C#-olioiksi.	Dataflow-kuvaus + koodi	☐
Arvioi ohjelmiston tietoturvaa	Tunnista tallennus-, syöte- ja salaisuusriskit; validoi data ja rajaa nimimerkki.	Riskit ja ratkaisut	☐
Käyttää versionhallintaa	Käytä Gitä koko ajan: selkeät commitit, pushit ja toimiva main.	Repository-historia	☐
Liittää ohjelman osan olemassa olevaan versioon	Toteuta palautemuutos branchissa ja yhdistä se testattuna mainiin.	Merge / pull request	☐
Julkaisee ohjelman tuotantoympäristöön	Tee Unity WebGL v1.0 ja julkaise se GitHub Pagesiin repositoryn docs-kansiosta.	Julkaisulinkki	☐

9. Dokumentointipohjat täytetään työn aikana

Dokumentointipohjan osa	Milloin	Mitä siihen tulee
1. Näyttöprojektin suunnitelma	Vko 35	Tavoite, asiakas, tarve, teknologiat, roolit, ominaisuudet, prioriteetti ja valmis kun.
2. Asiakastarve ja katselmoinnit	Aloita vko 34; päivitä vko 41 ja 47	Kysymykset, vaatimukset, tekninen viestintä, palaute ja päätetyt muutokset.
3. Tehtävälista ja työn suunnittelu	Päivitä viikoittain	Issue-linkki, prioriteetti, arvio, tekijä, tila ja toteuma/huomio.
4. Tekninen ja käyttöliittymäsuunnitelma	Ensiversio vko 35; päivitys vko 40	CafeGame-scene, Canvas-paneelit, C#-vastuut, mockup, TextAsset + JSON, PlayerPrefs, rajapinnat ja tietoturva.
5. Versionhallinta ja yhteistyö	Aloita vko 34; tarkista vko 47	Repository, branch-/merge-käytäntö, tehtävistä sopiminen ja katselmoinnit.
6. Testaus- ja virheenkorjausloki	Aloita vko 36; päivitä joka viikko	Päivä, kohde, testi/virhe, odotettu, tulos/syy, korjaus ja uusintatesti.
7. Julkaisu ja luovutus	Vko 48	Versio, julkaisu ympäristö, buildin tekeminen, tarkistukset, tunnetut puutteet ja käyttöohje.
8. Itsearviointi ja näytön yhteenvedo	Vko 49	Valmistunut työ, onnistumiset, vaikein ongelma, yhteistyö, palaute, opit ja seuraava kehityssaskel.

Viimeinen viikko päivä päivältä

Päivä	Päätavoite	Valmis kun
Ma 30.11.	Koodijäädytys	Tee viimeinen hyväksytty build. Uusia ominaisuuksia ei enää lisätä.
Ti 1.12.	Dokumentit ja todisteet	Täytä puuttuvat kohdat, testiloki, katselmointiloki, AI-loki ja linkit.
Ke 2.12.	Itsearviointi ja demon harjoittelu	Harjoittele 8-10 minuutin esitys ja valitse yksi bugi sekä yksi tekninen ratkaisu selitettäväksi.
To 3.12.	Puskuripäivä	Testaa julkaisulinkki, repositoryn näkyvyys ja palautuspaketti toisen henkilön kanssa.
Pe 4.12.	Lopullinen luovutus	Palauta peli, repository, dokumentit ja pidä sovittu näyttö. Projektin pitää olla valmis.

Luovutuksen tarkistuslista

- ☐ Julkaistu peli ja toimiva linkki
- ☐ Git-repository, README, järkevä commit-historia ja vähintään yksi merge/pull request
- ☐ Täytetty Näytön dokumentointipohjat -työpaketti
- ☐ Mockup ja tärkeimmät suunnittelu-/ratkaisumuistiinpanot
- ☐ Katselmointiloki ja asiakkaan palautteen perusteella tehty muutos
- ☐ Testaus- ja virheenkorjausloki sekä julkaistun version testaus
- ☐ Tekoälyn käyttöloki ja valmius selittää projektissa käytetty tekoälyapu
- ☐ Täytetyt osaamisvaatimusten näyttömatriisit ja itsearviointi